

Next-Gen AI Compiler

Agents On The Control Plane

Agents Own Search · Orchestration · Synthesis.
Compilers Own Lowering · Legality · Measure · Fallback.

Ask: Is The Hybrid Prediction (~2027-31) The Right Bet For Your Roadmap?

Expert Briefing · ~25-30 min + Q&A · Prediction-first

Agenda

- 
1. Trends · Keep Substrate
 2. Verdict + Four Jobs
 3. Architecture Evolution
 4. Prediction Checkpoints
 5. Commercial Blockers + Gaps
 6. Stance · Checklist · Ask

Trend backdrop — two stacks converging



Compilers for AI
mature · fragmented substrate

AI for compilers
rapid hybrid growth

Next-gen $\neq$ throw away LLVM/MLIR — make them **agent-addressable**.

Six active trends

A — Hybrid guidance

Free IR rewrite fragile
(mlirAgent < identity)
Constrain actions: hints /
advisory metadata / pass lists
AgentCompile · HintPilot · LLM Com-
piler

B — RL → agents

CompilerGym → tool agents
Heuristics as shippable C++
Workflow compile · freeze · place
Compiler-R1 · Magellan · FlowCompile

C — MLIR + Triton

PyTorch → Inductor → Triton
Also StableHLO → XLA / IREE
Peak often vendor libs / Tile
MLIR shared; product paths diverge

D — Kernel agents

KernelBench: correct *and* faster
One-shot often <20%; fusion hard
GEAK · KernelLLM · AgentCompile
Refine ≠ always faster

E — Verify in the loop

Unit/golden → numerical
→ Alive2-class local formal
Weak on GPU races / FP noise
Stacked oracle ladder for admit

F — Broader object

Mid-decode diagnosis
FMware: prompts / agents / knobs
Agents as compiler engineers
Compiler.next · Magellan · Claude C

Keep substrate, change control plane

Preserve

deterministic lowering
library kernels
build-CI regressability
local formal checks

Adopt

advisory search
multi-agent orchestration
heuristic synthesis
profile / verifier feedback

**Anti-pattern: replace opt/Inductor
with unconstrained LLM codegen**

Executive verdict

Agents reshape the **control plane** more than they replace the **data plane**.

Control plane = search / decide layer

Agents own: search, orchestration, synthesis

propose → measure → admit

Data plane = lower / execute layer

Compilers own: lowering, legality, measure, fallback

Inductor / MLIR / Triton stay the default path

Same stack, different jobs — agents above classical lowering, not instead of it.

Compilers for AI
× AI for compilers

Hybrid LLM-compiler loops

4th job Tier A:
bring-up / codesign

Will not happen soon

- Unconstrained LLM replaces Inductor / opt
- Autonomous chip tape-out via compiler agents (C10)

Four agent jobs — architecture spine

(a) Online

propose →
measure
admit

CompileIQ / GEAK
ACCLAIM / HintPilot

(b) Offline

evolve heuristics
→ C++ / MLGO

Magellan
AlphaEvolve

(c) Eng.

oracle-gated
pull request
/ change

Archer

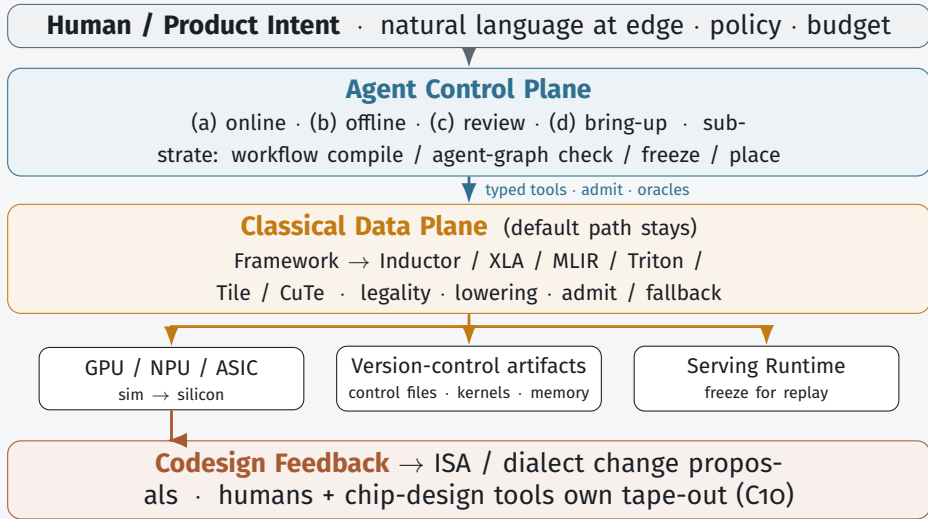
(d) Bring-up

coverage →
performance
sim + silicon

TritorX
KernelEvolve

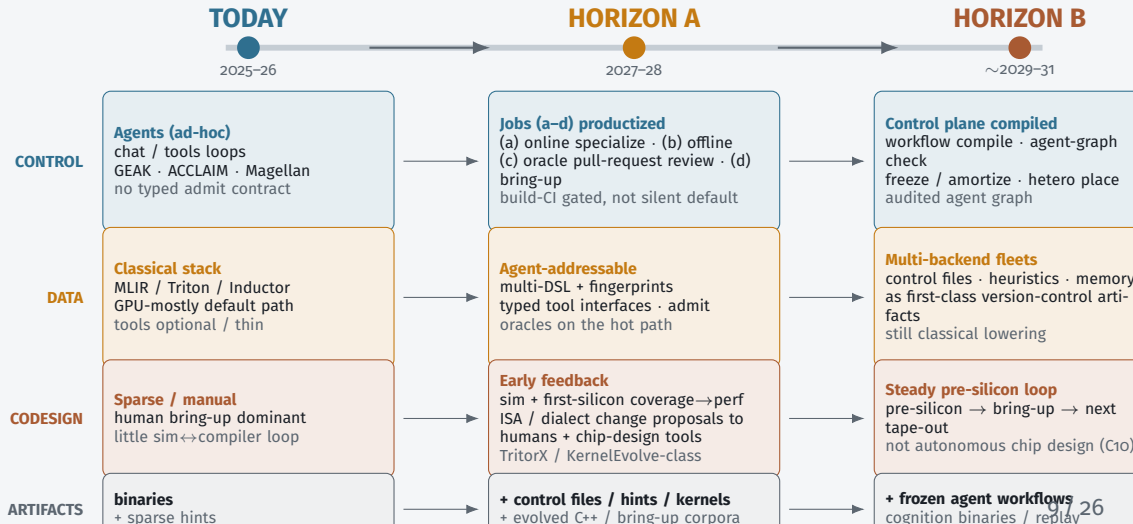
Control plane jobs sit *above* classical lowering — not instead of it.

Architecture — Target Stack



Invariant: LLM guides search — does not silently define unchecked executable behavior.

Architecture evolution — component changes



What ships / does not by 2028

Predicted to ship

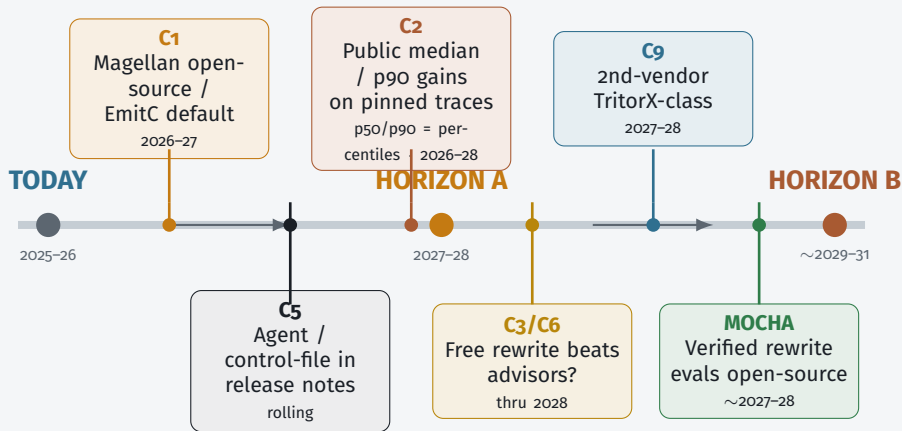
- Agent-addressable tool interfaces
- Hot-path specialize (not silent default-all)
- Magellan *and* MLGO both live
- Oracle pull-request review in serious orgs
- Coverage-first ASIC bring-up
- Triton-family primary; multi-DSL rising

Does not ship

- Unconstrained LLM replaces opt/Inductor
- One agent IR for all vendors
- Kernel agents uniformly beat eager on fusion ladders
- Autonomous microarch tape-out

Horizon A success = build-CI gated specialize + oracles + freeze artifacts

Roadmap Checkpoints — when the prediction changes



Falsifiable milestones — watch settlement signals; do not average disagreements.

Checkpoint C1 — heuristics vs neural advisors

A — Magellan path

evolve shippable C++ heuristics

Evolve shippable C++
(EVOLVE-BLOCK)

Offline coding agent
is the control-plane output

Product = evolutionary coding agent +
review

B — MLGO / EmitC path

MLGO = learned in-tree LLVM advisors

NN advisors stay in-tree

June 2026 plan of record:

internal inliner →

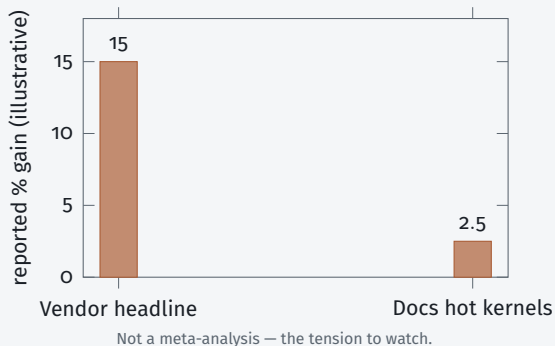
Android/Fuchsia →

Chrome multi-model

Agents train/feature NNs

Settlement: public Magellan llvm patches that dis-
place MLGO or EmitC-MLGO becomes customer default
Watch LLVM syncs + OpenEvolve recipes quarterly through 2027

Checkpoints C2 + C5 — gains & default path



C2 “Agents become default” needs **median (p50) build-CI wins**, not best-kernel blogs.

p50 / p90 = 50th / 90th percentile of the gain distribution across builds.

C5 Online specialize (control files / hints) vs offline eng (Magellan/MLGO) — both may live; *which is the default flag?*

Settlement: pinned public traces + release notes listing agent / control-file workflows.

Checkpoints C₃ / C₆ / C₉ / C₁₀ — interface width & scope

C₃ — free rewrite vs advisory

Flip: shared suite where free rewrite wins

⇒ wide rewrite interface vs narrow control files / hints

C₉ — coverage vs peak

Flip: TritorX → KernelEvolve ladder public

⇒ SKU: coverage service target then performance target

C₆ — replace vs control plane

Flip: production default with *no* admit

⇒ agents replaced the compiler (we reject this)

C₁₀ — codesign vs auto tape-out

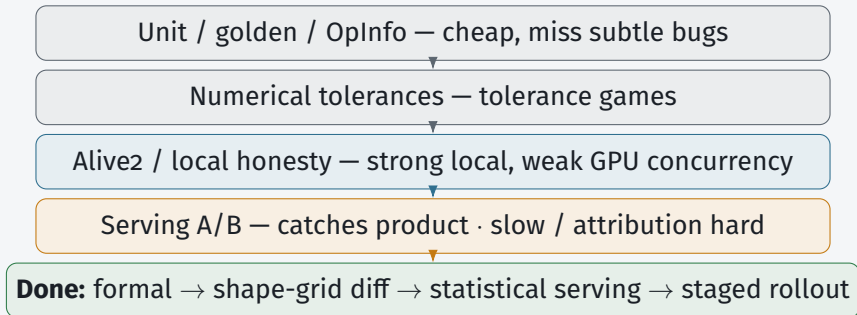
Flip: microarch primarily agent-proposed *and* compiler-oracle validated ⇒ enters chip-design tooling

Commercial blockers — top 5

- 1 Oracles strong enough for money
Fast-but-wrong · liability
- 2 Cost / replay / when agents run
Nondeterministic \$N compiles
- 3 Agent↔compiler contract
Natural-language-only = demo
- 4 Distributional production evidence
No default-path A/B
- 5 Ownership · supply chain · human review
Unowned agent code in trusted base

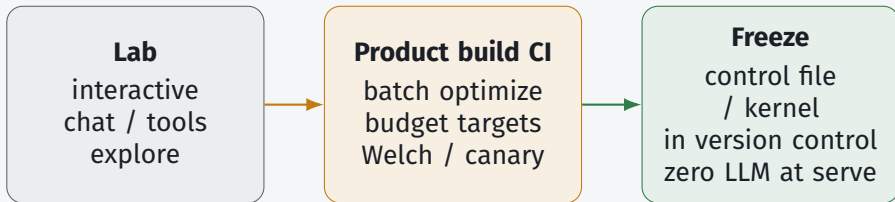
Each blocker gets one slide — productization gaps, not research wishlist.

Blocker 1 — Oracles for money



Room question: who owns false negatives when admit passes and production miscompiles?

Blocker 2 — Cost, replay, when may the agent run?



Cache key: (IR hash, hardware, compiler ver, agent policy) · budget \$/%gain

Falsifies commercially: “chat with compiler on every build” without spend/latency targets.

Blocker 3 — Agent $\leftrightarrow$ compiler contract

A. Natural language + pasted logs — demo only

B. Structured admit traces — build CI / bill of materials

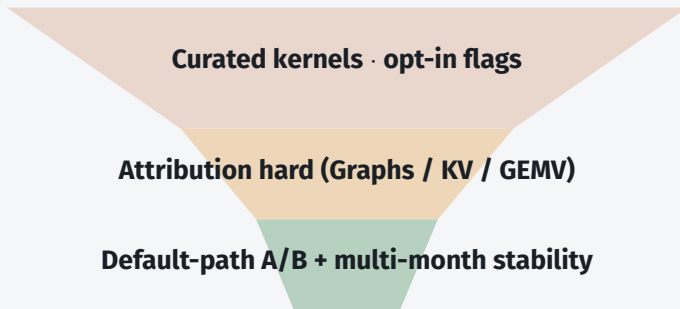
C. Typed tool interfaces — required action space

D. Hybrid — natural-language view over traces

```
admit_record = {graph_hash, hw_id, compiler_ver, action[], oracle[],  
artifact_digest, policy_id}
```

Lean: **C + B**; Natural language is a view, not source of truth. Advanced Control File = portable compiler knobs.

Blocker 4 — Distributional production evidence



Marketing bar: median + 90th-percentile gains + cost-to-compile
· persistent serving throughput, not single-kernel headlines

Blocker 5 — Ownership, security, human review

Trusted-base risk

agent kernels /
heuristics enter
trusted base

Ownership

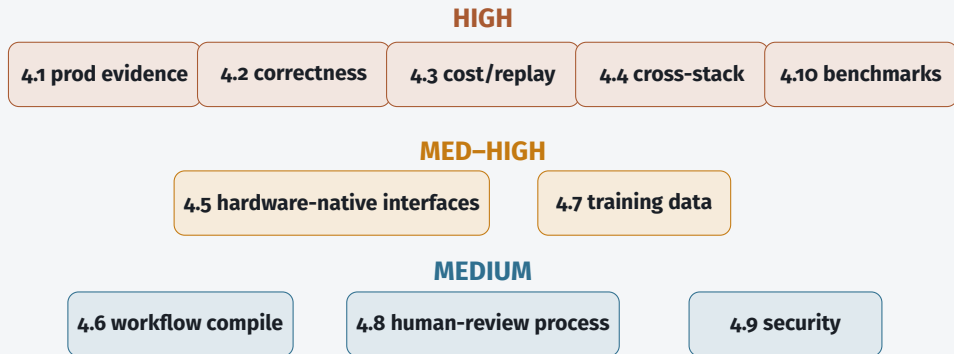
CODEOWNERS
signed provenance
sandbox

Human-review capacity

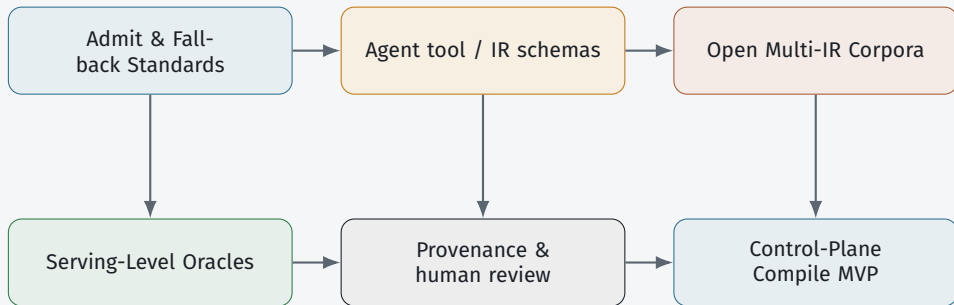
agents $\uparrow$ drafts
reviewers =
bottleneck

Lean: human CODEOWNER + signed admit + sand-
box · oracle auto-merge only for **narrow** action classes
Generic forge AI $\neq$ compiler-oracle review (C7)

Gap map — what blocks the prediction, not a wishlist



Cross-Cutting Research Agenda



Near-Term: Specs That Make Hybrid Pipelines Build-CI Regressable.

Org Adoption Questions

1. Online (CompileIQ) vs Offline (Magellan) — which matches your release model?
2. Oracle stack: Alive2 / golden / serving A/B — who owns false negatives?
3. Agent contract surface: LLVM / MLIR / Triton / StableHLO / Tile?
4. How do you cache/regress traces across compiler *and* model upgrades?
5. Max \$/build for a median X% win? Named maintainer per admitted artifact?

Working stance until conflicts settle

- 1 Hybrid: agents search; compilers lower (not replace)
- 2 Magellan || MLGO — parallel bets (C1)
- 3 Discount single-number speedups (C2)
- 4 Constrained actions + strong oracles (C3)
- 5 Demote generic SCM AI (C7)
- 6 Coverage→performance ladder; no autonomous chip design (C9/C10)

Commercial checklist — handout

Architecture

typed tools + admit
traces
version control $\gg$ dense
memory $\gg$ scratchpad
state-machine plan for
service targets
freeze before default-on

Trust

layered oracles
CODEOWNERS + prove-
nance
joint version pins
A/B before “prod default”

Business

sell regressable artifacts
+ build-CI quota
customer owns outputs
coverage then perfor-
mance service target
token budget per %gain

Discussion

Thank you — hybrid bet stays the ask

1. Which checkpoint flips your investment first — C1, C2, or C9?
2. Harder commercial bar: **oracle strength** or **replay/freeze**?
3. Building job (a), (b), or (d) first — what do you refuse online?

Ask again: is the hybrid bet right for your roadmap? Watch settlement signals — do not average disagreements.